FULL-STACK AI ENGINEER · DAY ASSIGNMENT

Nova Platform · GoComet

Build the intelligence. Then make it work for someone real.

Role	Full-Stack AI Engineer
Format	Two-part Day At Work Assignment (DAW), gated
Part 1	Multi-Agent Trade Document Pipeline · 8–16 hours
Part 2	Apply It to a Real CG Workflow · 3–4 hours · unlocked after Part 1 demo is reviewed
Evaluation	Tech 60% · Product & Outcome thinking 40%. Each part scored separately, then combined.

How This Assignment Works

This is a two-part assignment, **and Part 2 is gated.** You complete Part 1 and submit your demo. We review. If your Part 1 clears the bar, we send feedback and unlock Part 2. Part 2 builds directly on what you shipped in Part 1 — you are not rebuilding anything, you are extending it into a real workflow.

Part 1 — Build the Foundation

You build a multi-agent system that takes a trade document, extracts what matters, validates it against rules, and decides what to do next.

This shows us: can you design and ship an agentic system end-to-end? Do you make sharp tech choices and defend them? Do you understand what Nova actually is?

Time: 8–16 hours

Part 2 — Apply It to a Real Workflow

You take what you built and wire it into the real CG validation flow — a messy, human, exception-filled process where 4-cycle email loops are normal.

This shows us: do you understand the user, not just the model? Can you make agentic output usable for a non-technical operator?

Time: 3–4 hours · *unlocked after Part 1 review*

Part 2 is not optional and not independent. You must complete Part 1 first and pass the gate. What you ship in Part 1 is the raw material for Part 2. If Part 1 doesn't clear the bar, we'll tell you why — and you won't be asked to do Part 2.

PART 1 · 8–16 HOURS · Build the Multi-Agent Foundation

Context — The Problem You're Solving

In global trade, every shipment generates a stack of documents — Bill of Lading, Commercial Invoice, Packing List, Certificate of Origin. These docs are emailed around as PDFs and scans, read field-by-field by humans, and validated against customer-specific rules that mostly live in someone's head.

Most companies treat this with a giant pile of human attention. We think a multi-agent system can do the boring 80% — extract, validate, decide — and let humans handle only the exceptions.

We want to test this product idea by building three agents that work together:

- **Extractor Agent —** takes any trade document (PDF or image) and pulls out structured fields using a vision-capable LLM.
- **Validator Agent —** compares the extracted fields against a rule set (customer requirements) and produces a field-by-field result with a confidence score.
- **Router / Decision Agent —** decides what to do next based on the validation: auto-approve, flag for human review, or draft an amendment request.

> **Why three agents and not one giant prompt?** That is exactly the kind of question we want you to think about and answer in your PRD. There is a right answer, but we want your reasoning, not a guess.

Part 1 — What to Build

Three deliverables. In this order.

Deliverable 1 — PRD (3–5 pages)

Execution-oriented. Not a vision doc. An engineer should be able to read this and start building.

1 | Show us you understood Nova

If you are a junior or an intern, slow down here. Read the JD. Read this brief. Then in your own words (max 200 words each), explain:

- What is Nova? What problem is it solving that traditional SaaS can't?
- What is the FDE (Forward Deployed Engineer) model and why does GoComet use it for Nova?
- What does "System of Outcomes" mean? How is it different from "System of Record" or "System of Engagement"?

If any of these terms are new, that's fine. Read the JD twice, search around, ask Claude — but write your own answer. We can spot a copy-paste from a mile.

2 | Problem Statement

- Where does the current trade-doc validation flow break? Be specific — name the failure modes.
- What does success look like for a CG operator in their first 5 minutes of using your system?

3 | Users + Jobs-to-be-Done

- At least 2 personas — one operator (CG), one supplier (SU).

- At least 5 JTBD statements — clear, testable, in the form: When ___, I want to ___, so that ___.

4 | Agent Architecture (this is the technical core — be sharp here)

- Why three agents? Why not one prompt? Why not five? Defend the boundary.
- What is each agent's responsibility, input, and output? Use a planner / executor / verifier framing if it fits.
- How do the agents talk to each other? (Shared memory, message passing, structured handoff?)
- How does state survive a crash mid-pipeline?

5 | LLM & Tooling Choices (defend every pick)

- Which LLM(s) for which agent? Why? What are the cost / latency / quality tradeoffs?
- Which vision model for extraction? What's your fallback when the doc is bad quality?
- Which framework for orchestration (LangGraph, custom, something else)? Why?
- Where do you use structured output / function calling / tool use? Where do you avoid it?

6 | Trust, Failure Handling & Evals

- How do you stop the agent from hallucinating a field that wasn't in the doc?
- How do you handle low-confidence extractions? (Hint: silent approval is the worst possible answer.)
- How do you stop agent loops, runaway costs, and infinite retries?
- How would you eval this system? Define at least one offline eval and one online metric you would actually run.

7 | Metrics & Success Criteria

- 1 north-star metric. One number. One sentence.
- 5–8 supporting metrics — mix of agent quality, system health, and business outcome.
- Go / No-Go criteria for a 2-week pilot with one customer.

8 | What's Next (after Part 1 ships)

- If you had two more weeks, what would you build next? Why that, and not something else?

Deliverable 2 — Working POC

Three behaviours below are the minimum bar. Partial does not pass.

A | Extractor Agent (required)

Accepts any trade document (PDF or image). Uses a vision-capable LLM. Outputs structured JSON with at least these fields: consignee name, HS code, port of loading, port of discharge, Incoterms, description of goods, gross weight, invoice number. Each field must include a confidence score, not just a value.

B | Validator Agent (required)

Takes the extracted JSON and a rule set (which you define for one customer). Produces a field-by-field result: match, mismatch, or uncertain. Mismatches must include what was found vs what was expected. Uncertain fields must surface — never silently approve.

C | Router / Decision Agent (required)

Reads the validator's output and decides one of three outcomes: (1) auto-approve and store, (2) flag for human review with reasoning, or (3) draft an amendment request listing each discrepancy. The agent must explain its decision, not just emit it.

D | Storage + Query (required)

Verified outputs are stored in a queryable form (SQLite, DuckDB, ClickHouse, Postgres — your call). A non-engineer should be able to ask basic natural-language questions over the stored data — e.g. "how many shipments were flagged this week?" — and get a grounded answer. The query layer can be simple. The chain matters more than the polish.

E | Minimal UI (required)

A simple screen that shows the pipeline running on one document: extracted fields, per-field confidence, validation result, decision, and the agent's reasoning. Does not need to be pretty. Must show real state from a real run.

Deliverable 3 — Technical Write-up (1–2 pages)

This is where you separate from candidates who can only build but can't think about what they built.

- One architecture diagram — boxes, arrows, where data flows, where state lives.
- How you handle the three nastiest failure modes you can think of. Real examples from your own testing, not hypotheticals.
- Observability — if this was running in production for 50 customers, how would you trace a single shipment from email to verified output? What would your dashboard show?
- Cost — back-of-envelope cost per document. Where does it blow up? How do you control it?
- Latency — where is the slowest hop? What would you do to fix it?
- What you would do differently if you had a week instead of a day.

Part 1 — Submission

- A single repo (or zipped project) containing the runnable POC.
- README with clear setup and run instructions. We must be able to run it on a laptop.
- PRD as a PDF or Google Doc.
- Technical write-up as a PDF or Google Doc.
- At least 2 sample documents you tested on (one clean, one messy / low-quality).
- A 2–3 minute demo video walking through the pipeline on one document.
- Sample queries you ran against the stored output.

Part 1 — Evaluation

Tech-heavy weighting (60% tech, 40% product & outcome thinking). The job is half engineer, half consultant — so we score both, but tech leads.

Dimension	What we're looking for	Weight
Architecture & code quality	Sharp agent boundaries, clean handoffs, defensible tech choices, code that doesn't make us cry.	20%
AI craft	Hallucination handling, confidence surfacing, eval thinking, cost & latency awareness, observability story.	20%
End-to-end demo	All five behaviours (A–E) actually run on real input. The chain is alive.	20%
PRD depth & Nova understanding	Clear problem framing, real JTBDs, credible metrics, AND proof you understood Nova / FDE / System of Outcomes in your own words.	20%
Outcome & product thinking	North-star is testable. Failure modes are real. Trust handling is explicit, not vibes.	15%
Communication	Tech write-up is sharp. Demo video is tight. README does not waste our time.	5%

PART 2 · 3–4 HOURS · Apply It to a Real Workflow

Reminder: Part 2 unlocks only after we review your Part 1 demo. The brief below is included so you know what you are working toward — but do not start it until we send the go-ahead and feedback on Part 1. The story below is just an example; it will change after part 1 feedback.

The Story — What's Actually Happening

Read this before you write a line of code for Part 2. This is the real workflow your solution has to fit into.

The Three People in This Workflow

Who	Role	What they care about
SU (Shipping Unit)	The supplier / shipper	Goods are dispatched. Documents generated — Bill of Lading, Commercial Invoice, Packing List, Certificate of Origin. Their job feels done once the email is sent.
CG (Cargo / Control Group)	The validator	Receives SU's docs, cross-checks every field against what the customer requires, and replies: approved, or here's what needs fixing.
Customer	The end recipient	Gets one clean, correct document set. A wrong HS code or mismatched consignee means customs delays, cargo holds, or a contract penalty.

What Happens Today

SU generates the shipment documents and emails them to CG. CG opens every attachment, reads each field manually, and checks it against what the customer requires. If something is wrong — wrong consignee name, incorrect HS code, missing Incoterm — CG types out an amendment request and sends it back to SU. SU fixes it and resubmits. This loop repeats. Two to four cycles per shipment is normal. Once CG is satisfied, the approved docs go to the customer.

What CG actually does all day	**Where time and accuracy get lost**
• Opens email, downloads attachments • Reads every field in every document • Mentally checks against customer rules • Types out what's wrong, field by field • Sends amendment email back to SU	• Rules for each customer live in people's heads • A new CG hire makes mistakes for weeks • No visibility: how many docs are pending?

• Waits. Checks again. Repeats.	• Each amendment cycle adds 4–24 hrs of delay • No audit trail if a dispute arises later • CG bandwidth limits how fast shipments clear

The process is correct. SU sends, CG validates, customer receives — that three-party structure stays. What doesn't need to exist is a human reading every field and typing every amendment manually.

Where This Connects to Part 1

In Part 1, you built three agents — Extractor, Validator, Router — and a query layer over verified output. Those four pieces are exactly what CG needs.

What's missing is the trigger. Today, the agent only runs when you upload a doc to a screen. In Part 2, the agent has to wake up the moment SU's email arrives, read the attached doc(s), validate them against the customer's rules, and hand CG a verification result and a draft reply — instead of an inbox attachment.

You are not rebuilding anything. You are connecting what you built to a real workflow.

Part 2 — What to Build

Three deliverables. In this order.

Deliverable 1 — PRD (max 1 page · ~30 min)

Tight and scannable. A CG team lead should read this in 3 minutes and know what the agent does for them.

Cover exactly these — nothing more

- 2 personas — one CG, one SU. What does each actually care about in this workflow?
- 2 JTBDs — "When ___, I want to ___, so that ___." Specific to this workflow.
- 1 north-star metric — is CG validation getting faster and more accurate? Pick one number.
- 1 failure mode — what is the worst thing the agent could do, and how does it stop that?

Deliverable 2 — UI Module (~45 min)

Build a working screen — like you would in Lovable, v0, or any rapid prototype tool — that shows the CG user experience. Not a diagram. Clickable. Real states.

The screen must show these four states

- Incoming — new email from SU just arrived, doc attached, agent is processing.
- Verification result — field-by-field view: which matched, which didn't, confidence per field.

- Discrepancy detail — clicking a flagged field shows what was found vs what was expected, plus the source snippet from the doc.
- Draft reply — the agent's generated email to SU, editable before CG sends it. Agent never sends on its own.

Tech stack: your choice. React, plain HTML, Streamlit, whatever you build fastest in. The point is that a CG operator can look at this and understand what the agent found in 10 seconds.

Deliverable 3 — Working Wiring (~1.5–2 hours)

Connect your Part 1 agents to the UI above and to a simulated SU inbox. Must run end-to-end.

Step	What to build	What we're checking
1	Trigger — SU's email (or file) arrives. Watch a folder or simulate an inbox. When a new email with attached docs appears, the pipeline activates. Mock the email plumbing — the logic is what matters.	Did you understand that the trigger is the missing piece, not the model?
2	Extract — Reuse the Extractor Agent from Part 1. Handle the case where there are multiple attachments per shipment.	Does multi-doc handling work, or did you assume one doc per email?
3	Cross-validate — When a shipment has 3 docs (BOL + Invoice + Packing List), fields like consignee and HS code must match across all three. Add this check.	Do you understand cross-document consistency, not just per-doc validation?
4	Decide & Draft — Router Agent produces either a clean approval email or an amendment email listing every discrepancy with field name, found, expected.	Is the draft something a CG operator can send with one edit, or does it need a rewrite?
5	Hand off — Verified output is stored and queryable via your Part 1 query layer. CG can ask "show me everything pending review for customer X."	Is the chain from email to query alive?

Agent never sends on its own. CG always reviews and clicks send. This is non-negotiable. We do not want a system that emails customers without human review on day one.

Part 2 — Suggested Time Split

Time	Task	What we're watching
0:00–0:30	Read brief + write the PRD	Do you understand the user, not just the tech?
0:30–1:15	Build the UI module — CG verification screen	Can you translate user behaviour into a real interface?
1:15–3:00	Wire trigger → extractor → cross-validation → router → draft	Core pipeline quality, failure handling, multi-doc reasoning
3:00–3:30	End-to-end test on 2 shipments (one clean, one messy)	Is the full chain alive and usable?

Part 2 — Evaluation

Dimension	What we're looking for	Weight
User behaviour thinking	Do the PRD, the UI, and the wiring reflect how CG and SU actually work? Is the no-process-change constraint respected?	30%
Working pipeline	Trigger to draft runs end-to-end on a real shipment. Multi-doc cross-validation actually fires.	25%
UI quality	A non-technical CG operator can use this in 10 seconds. Discrepancy detail is genuinely useful.	20%
Trust & failure handling	Agent surfaces uncertainty. Never sends. Fails loud, not silent. No silent approvals.	15%
Outcome thinking	North-star is specific enough that a CG team lead can tell on Day 14 whether it's working.	10%

Part 2 — Submission

- Runnable agent + UI — README with setup and run instructions.
- PRD — PDF or Google Doc, max 1 page.
- Sample SU emails + trade documents used in your demo.
- A 2-minute demo video showing the trigger → verification → draft → CG-edits-and-sends loop.

Part 1: build something that works.

Part 2: make it matter to someone real.